

Static & Dynamic Testing Techniques

Статичні та динамічні техніки тестування програмного забезпечення

Автор: Анна Гуртова (Anna Hurtova) Курс: Beetroot Academy / Helvetas Ukraine ("Стійкість")
Роль: Trainee QA Manual Engineer Дата: Липень 2026

Блок 1: Порівняльний аналіз статичних та динамічних технік

КРИТЕРІЙ	СТАТИЧНА ТЕХНІКА (STATIC)	ДИНАМІЧНА ТЕХНІКА (DYNAMIC)
Головна концепція	Перевірка робочих артефактів проекту без безпосереднього запуску коду програми.	Обов'язковий запуск коду програми на виконання та взаємодія з активною системою.
Об'єкт тестування	Технічна документація, вимоги (ТЗ), макети дизайну, архітектура системи та вихідний текст коду.	Працюючий графічний інтерфейс користувача (UI/UX) та функціональні модулі софту.
Час застосування	Починається на ранніх етапах (Requirements Analysis) до появи першої збірки продукту.	Застосовується на етапі готовності робочого функціоналу.
Методологічна основа	Статичний White-Box на предмет верифікації: структурний аналіз внутрішнього улаштування коду.	Динамічний Black-Box та White-Box на предмет валідації: поведінковий аналіз на основі специфікацій + структурна перевірка покриття.

Використовувані техніки та методи

СТАТИЧНІ ТЕХНІКИ	ДИНАМІЧНІ ТЕХНІКИ
Формальні та неформальні перегляди (Reviews): — Technical Review (оцінка архітектури колегами) — Code Review / Peer Review (статичний White-Box) — Requirements Review (пошук неточностей у ТЗ) — Design Inspection (перевірка логіки дизайну) — Walkthroughs (наскрізний розбір алгоритмів) — Formal Inspection (аудит за чек-лістами)	Black-Box (Специфікаційні): — Equivalence Class, Boundary Value Analysis — Decision Table, State-Transition — Use Case Testing, Pairwise, Domain Analysis
Автоматизований статичний аналіз: — Linter-перевірки (синтаксис та стиль коду) — Data Flow Analysis (неініціалізовані змінні) — Control Flow Analysis (недосяжний код)	White-Box (Структурні): — Statement Testing, Decision/Branch Testing — Path Testing Grey-Box (Інтеграційні): — Тестування API-ендпоінтів — Перевірка БД та серверних логів Experience-based: — Exploratory Testing, Error Guessing

Переваги та обмеження

#	СТАТИЧНІ ТЕХНІКИ	ДИНАМІЧНІ ТЕХНІКИ
ПЕРЕВАГИ		
1	Економічна ефективність: Локалізація помилок у вимогах до написання коду. Виправлення дефекту в документації коштує в десятки разів дешевше, ніж ремонт готового софту.	Валідація користувацького досвіду: Оцінка додатка у тому вигляді, у якому його побачить клієнт. Тільки в динаміці перевіряється стабільність верстки, плавність переходів та зручність UX/UI.

#	СТАТИЧНІ ТЕХНІКИ	ДИНАМІЧНІ ТЕХНІКИ
	Оптимізація архітектури: Code Review допомагає вчасно виявити заплутану логіку, ресурсомісткі цикли або приховані вразливості безпеки.	Комплексна перевірка інтеграцій: Тестування взаємодії фронтенду, бекенду та сторонніх сервісів (авторизація Facebook, платіжні системи).
3	Скорочення термінів: виправлення неточностей у документах на старті дає програмістам чітке розуміння завдань та мінімізує Rework.	Об'єктивність результатів: Результат завжди вимірюваний — система або виконала дію за ТЗ (Passed), або видала помилку (Failed).
ОБМЕЖЕННЯ		
1	Не здатний оцінити працездатність у реальному часі: навантаження на сервер, затримки БД, стабільність інтеграцій.	Неможливо застосувати до появи робочої збірки (build). Виявлення архітектурних помилок на цьому етапі вимагає серйозного рефакторингу.
2	Високий ризик людського фактора: ефективність вичитки залежить від досвіду та уважності інженера.	Фіксує лише зовнішній симптом збою. Потрібен додатковий аналіз логів та мережевих запитів для ізоляції першопричини.
3	Неможливо оцінити зручність інтерфейсу, поки елементи не рухаються. Дефекти анімації не видно у тексті коду.	Потреба у тестовій інфраструктурі: Staging-сервери, тестові БД та парк реальних пристроїв для крос-платформної верифікації.

Підсумок: Статичні та динамічні техніки не конкурують між собою, а є взаємодоповнюючими елементами єдиного процесу забезпечення якості. Статичний аналіз закладає стабільний фундамент на початку SDLC, а динамічний забезпечує фінальну валідацію перед релізом. Лише їхнє послідовне поєднання гарантує випуск конкурентоспроможного ІТ-продукту.

Блок 2: White-Box аналіз — Decision & Statement Coverage

Задача 1: Покриття рішень для коду з однією IF-умовою

Твердження: «Коли код має одну IF-умову, не має циклів (LOOP) або перемикачів (CASE), будь-який тест дасть результат 50% покриття рішень.»

✗ Коректно. Будь-який тест-кейс надає 100% покриття тверджень, таким чином покриває 50% рішень.

✓ Коректно. Результат будь-якого тесту умови IF буде або правдивим, або ні.

✗ Некоректно. Один тест може гарантувати 25% перевірки рішень.

✗ Некоректно, бо занадто загальне твердження.

Обґрунтування: Одна IF-умова має два можливих результати: TRUE або FALSE. Кожен окремий тест-кейс може пройти лише одну з двох гілок, що дорівнює 50% покриття рішень (Decision Coverage). Для 100% покриття потрібні мінімум 2 тести.

Задача 2: Псевдокод MS Word

Switch PC on → Start MS Word → IF MS Word starts THEN → Write a poem → Close MS Word

Скільки тест-кейсів знадобиться?

✓ 1 — для покриття операторів, 2 — для покриття рішень

✗ 1 — для покриття операторів, 1 — для покриття рішень

✗ 2 — для покриття операторів, 2 — для покриття рішень

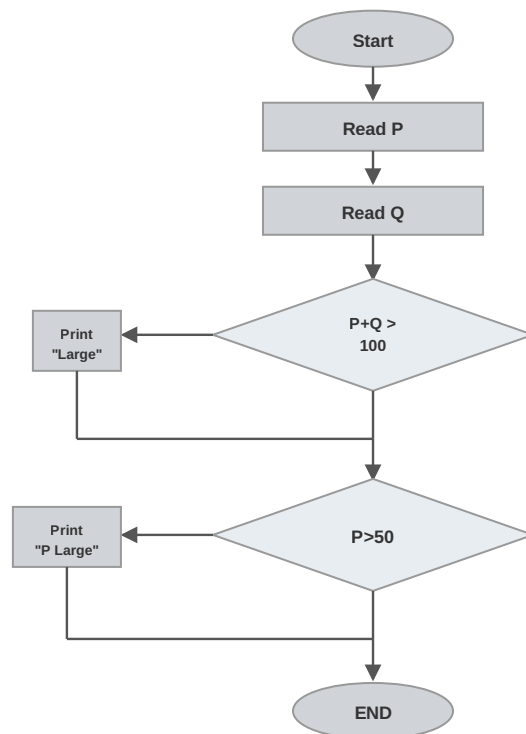
✗ 2 — для покриття операторів, 1 — для покриття рішень

Обґрунтування: Statement Coverage: 1 тест (IF = TRUE) проходить через усі оператори послідовно. Decision Coverage: потрібні 2 тести — один для TRUE (MS Word запустився → пишемо вірш), другий для FALSE (MS Word не запустився).

Задача 3: Покриття тверджень коду

Скільки потрібно тестів для перевірки тверджень коду:

```
Read P
Read Q
IF P+Q > 100 THEN
    Print "Large"
ENDIF
IF P > 50 THEN
    Print "P Large"
ENDIF
```



✗ a. 2

✓ b. 1

✗ c. 3

✗ d. 4

Обґрунтування: Для 100% Statement Coverage достатньо **1 тесту**, який виконає всі оператори коду. Наприклад, при P=60, Q=50: умова P+Q>100 = TRUE (друкується "Large"), умова P>50 = TRUE (друкується "P Large"). Один тест-кейс проходить через усі рядки — жоден оператор не залишається невиконаним.

Блок 3: Алгоритм опитувальника CatPaw Share

Мета: Оптимізація та перевірка логіки розгалуженого опитувальника. Визначення мінімальної кількості тест-кейсів для 100% покриття рішень та шляхів (Decision / Path Coverage).

Структурований псевдокод алгоритму

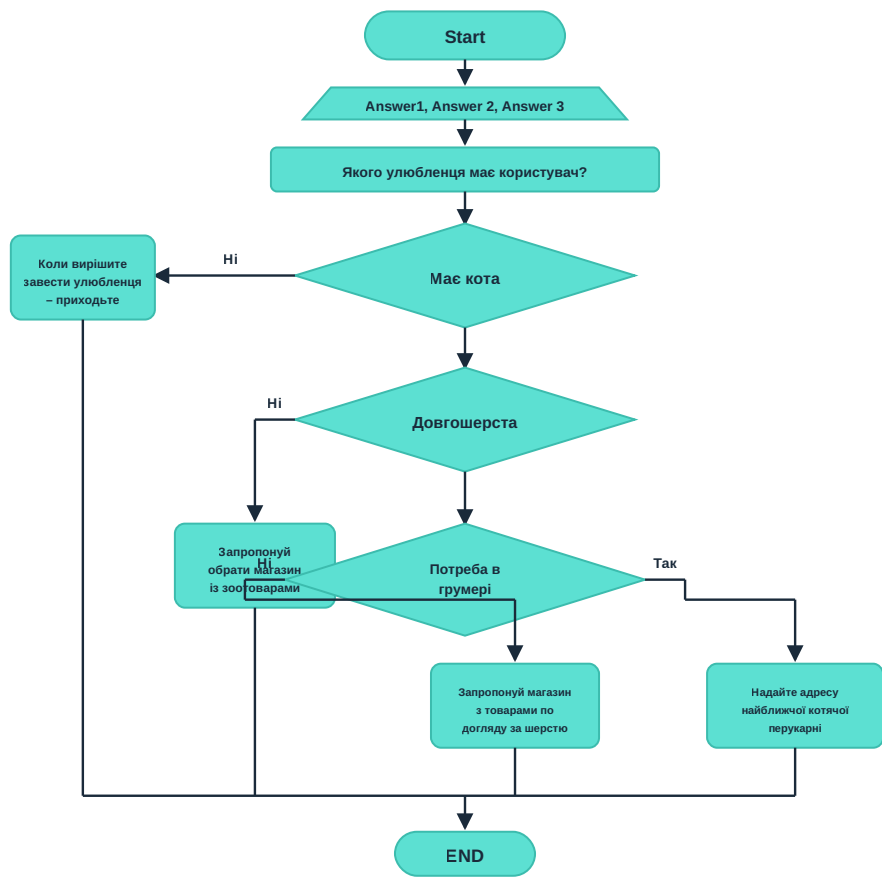
```
Input: Answer1 (Наявність кота), Answer2 (Довжина шерсті), Answer3 (Потреба в грумері)

Print "Якого улюбленця має користувач?"

IF Answer1 == "Має кота" THEN
  IF Answer2 == "Довгошерста" THEN
    IF Answer3 == "Так, потрібен грумер" THEN
      Print "Надайте адресу найближчої котячої перукарні"
    ELSE
      Print "Запропонуйте магазин з товарами по догляду за шерстю"
    ENDIF
  ELSE
    Print "Запропонуйте обрати магазин із зоотоварами"
  ENDIF
ELSE
  Print "Коли вирішите завести улюбленця – приходьте"
ENDIF

END
```

Блок-схема алгоритму (Flowchart)



Мінімальний набір тест-кейсів для 100% Path Coverage

Відповідь: Для перевірки всіх комбінацій необхідно **4 тест-кейси** — це відповідає найвищому рівню White-Box тестування (100% Path Coverage).

ТС	ANSWER1 (KIT?)	ANSWER2 (ШЕРСТЬ?)	ANSWER3 (ГРУМЕР?)	ОЧІКУВАНИЙ РЕЗУЛЬТАТ (ШЛЯХ)
ТС-01	Має кота	Довгошерста	Так	«Надайте адресу найближчої котячої перукарні»
ТС-02	Має кота	Довгошерста	Ні	«Запропонуй магазин з товарами по догляду за шерстю»
ТС-03	Має кота	Короткошерста	—	«Запропонуй обрати магазин із зоотоварами»
ТС-04	Не має кота	—	—	«Коли вирішите завести улюбленця — приходьте»